Behavioral pattern

A **behavioral pattern** is a software design pattern for collaboration between objects.

Examples

Examples include:

Blackboard design pattern
: Provides a computational framework for the design and implementation of systems that integrate large and diverse specialized modules, and implement complex, non-deterministic control strategies.

Chain-of-responsibility pattern
: Command objects are handled or passed on to other objects by logic-containing processing objects.

Command pattern
: Command objects encapsulate an action and its parameters.

Externalize the stack
: Turn a recursive function into an iterative function that uses a stack[1]

Interpreter pattern
: Implement a specialized computer language to rapidly solve a specific set of problems.

Iterator pattern
: Iterators are used to access the elements of an aggregate object sequentially without exposing its underlying representation.

Mediator pattern
: Provides a unified interface to a set of interfaces in a subsystem.

Memento pattern
: Provides the ability to restore an object to its previous state (rollback).

Null object pattern
: Acts as a default value of an object.

Observer pattern
: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. The variant **weak reference pattern** decouples an observer from an observable to avoid memory leaks in environments without automatic weak references.[2]

Protocol stack
: Communications are handled by multiple layers, which form an encapsulation hierarchy[3]

Publish–subscribe pattern
: A messaging pattern where senders (publishers) and receivers (subscribers) are decoupled via message topics and brokers. Commonly used in distributed systems, this pattern supports asynchronous, many-to-many communication.

Scheduled-task pattern
: A task is scheduled to be performed at a particular interval or clock time (used in real-time computing).

Single-serving visitor pattern
: Optimise the implementation of a visitor that is allocated, used only once, and then deleted.

Specification pattern
: Recombinable business logic in a boolean fashion.

State pattern
: A clean way for an object to partially change its type at runtime.

Strategy pattern
: Algorithms can be selected on the fly, using composition.

Template method pattern
: Describes the skeleton of a program; algorithms can be selected on the fly, using inheritance.

Visitor pattern
: A way to separate an algorithm from an object.

See also

- Concurrency pattern
- Creational pattern
- Structural pattern

References

1. "Externalize The Stack" (https://web.archive.org/web/20110303085751/http://c2.com/). c2.com. 2010-01-19. Archived from the original on 2011-03-03. Retrieved 2012-05-21.
2. Nakashian, Ashod (2004-04-11). "Weak Reference Pattern" (https://web.archive.org/web/20110303085751/http://c2.com/). c2.com. Archived from the original on 2011-03-03. Retrieved 2012-05-21.
3. "Protocol Stack" (https://web.archive.org/web/20110303085751/http://c2.com/). c2.com. 2006-09-05. Archived from the original on 2011-03-03. Retrieved 2012-05-21.